AUTOMATED FOOTBALL CARTOON REELS

Technical blueprint for a repeatable, AI-assisted 2D animation pipeline

Recommended architecture
Rigby2D for character rigging and authoring · Godot for headless animation and scene rendering · Python for orchestration · Rhubarb for lip-sync · FFmpeg for final encoding

Project requirements

- Vertical 9:16 videos at 1080 × 1920, typically 20–35 seconds and 4–6 scenes.
- Original bold-outline, flat-colour caricatures of real public figures, with reusable character identities.
- Real character motion: walking, bending elbows and knees, waving, pointing, holding props, blinking, talking and changing expressions.
- Daily production with minimal manual work; headless cloud execution on free or existing compute.
- Open-source tools preferred; no paid subscriptions or credit-card dependency.

Executive recommendation

Build a hybrid 2D skeletal-animation pipeline. Use AI to create character artwork and backgrounds, but use deterministic rigs and reusable action behaviours for the actual animation. Keep AI video generation as an optional tool for occasional special shots—not the foundation of the daily production system.

The core shift: stop treating each scene as an image to animate. Treat each character as a reusable digital puppet with a skeleton, face controls, outfits, props and a library of actions.

1. Proposed production architecture

Stage	Responsibility	Output
1. AI script generator	Writes the joke, scene sequence, dialogue, actions, timing and camera direction.	Validated scene JSON
2. Character asset factory	Creates and maintains consistent character artwork, body layers, expressions, outfits and props.	Reusable character packages
3. Animation engine	Executes bone movement, IK, facial controls, action blending and prop interactions.	Animated scene timelines
4. Scene compositor	Combines characters, backgrounds, camera movement, captions, speech bubbles and SFX.	Rendered frames / scene video
5. Automated renderer	Runs headlessly and encodes the final video.	1080 × 1920 MP4

Keep the animation engine separate from the video renderer. Python should orchestrate the production; it should not have to invent every joint movement or perform all animation math itself.

2. Choosing the animation technology

The four practical approaches

Approach	Strengths	Weaknesses	Fit
2D skeletal rig	Precise joint control, reusable cycles, consistent identity, prop interaction.	Requires initial character rigging and action-library work.	Best foundation
AI motion transfer	Can transfer full-body movement from reference footage; quick to experiment.	Pose/prop artifacts, less precise timing, variable consistency and commercial terms.	Optional experiments
AI image-to-video	Can produce expressive, cinematic movement from a still image.	Character drift, difficult shot-to-shot continuity, GPU cost and unpredictable output.	Special shots only
Hybrid	Rig handles repeatable actions; AI handles selected assets or unusual shots.	More components to integrate.	Best overall

A. 2D skeletal animation

A proper rig uses parent-child bones, weighted meshes, inverse kinematics (IK), facial controls and reusable animation timelines. Mesh deformation lets the elbow and knee bend rather than rotating a single rigid arm or leg segment.

- A stamping action can coordinate torso lean, shoulder movement, elbow bend, wrist rotation, hand grip, stamp impact and recoil.
- IK helps position hands and feet naturally, especially when interacting with a phone, document, desk or ground.
- Use overlapping artwork at joints so seams remain hidden during rotation and deformation.

B. Motion transfer: Viggle AI

Motion transfer takes character artwork plus a reference movement video and attempts to transfer the movement to the character. It can be useful for exaggerated running, dancing or reactions, but is not ideal as the core engine because individual joints, props, timing and identity are not fully deterministic.

Viggle's free-tier terms and commercial permissions should be checked before monetised use. Do not assume that a free web-service plan grants unrestricted commercial rights.

C. Open-weight video models

- Wan 2.2: capable open-weight video generation, but demanding on hardware and difficult to use for consistent multi-scene character animation.
- LTX-Video: worth testing for speed-oriented video-generation experiments; check the specific model, checkpoint and hardware requirements.
- ComfyUI: useful for reproducible node-based image/video workflows and model experiments.

A T4 generally has 16 GB VRAM. Expect compromises with larger video models: quantisation, CPU offloading, lower resolution, short clips and longer generation times. The exact feasibility depends on the checkpoint and workflow.

3. Tool selection and recommended stack

Tool	Role	Licence / notes	Recommendation
Rigby2D	Character rig authoring, AutoRig, meshes, IK and animation.	Project documentation identifies MIT / Apache-2.0 components; verify the exact release and dependencies.	Test first for character rigging
Godot 4.x	2D scene engine, skeletal playback, animation, camera, captions and headless rendering.	MIT licence; commercial use is permitted subject to licence notice requirements.	Recommended renderer
Blender	Python-driven 2D/3D animation, rigging and headless rendering.	GPL; artwork produced with Blender can be used commercially.	Alternative / future 2.5D
Rhubarb Lip Sync	Audio-to-mouth-shape timing for 2D characters.	MIT licence.	Recommended lip-sync utility
Python	Scene validation, orchestration, asset management and job control.	Python ecosystem; check each dependency separately.	Keep existing pipeline
FFmpeg	Video encoding, audio mixing and format conversion.	LGPL/GPL depending on build configuration and enabled components.	Keep existing pipeline
GitHub Actions / Kaggle	Automation and optional GPU asset generation.	Free allowances and session/resource limits apply.	Use within quotas

Why Godot?

- Native 2D scene management, Skeleton2D/Bone2D, Polygon2D, AnimationPlayer and AnimationTree.
- GDScript makes it possible to build a deterministic action player and scene director.
- Supports command-line and headless workflows; test the exact movie-output and frame-capture route in the intended CI environment.
- Can render PNG frames for FFmpeg encoding, which is a straightforward fallback if direct movie output is awkward.

Rigby2D integration caveat

Rigby2D is a promising authoring candidate, but do not assume the editor itself is a complete unattended command-line film renderer. Its documented exports and runtimes should be tested in a small Godot project. The editor's built-in Python sandbox is not a general external automation API. A practical fallback is to bake standard actions into sprite sheets and play them in Godot.

Recommended division of labour

Task	Tool
Create initial character design and background artwork	AI image generator
Separate and prepare layers; repair joints and transparency	Image editor / Python image processing
Build and tune the master rig	Rigby2D
Play actions and compose scenes	Godot
Validate JSON and schedule render jobs	Python
Generate mouth timings	Rhubarb Lip Sync

Task	Tool
Encode MP4 and mix SFX / beat	FFmpeg

The proposed stack does not require AI video generation for ordinary scenes. Use GPU time for new artwork and optional special shots; run the skeletal animation and 2D composition on CPU where practical.

4. Producing rig-ready character artwork

The current A-pose auto-cut approach fails because a flattened image does not contain the hidden geometry needed to bend at the elbow, knee and shoulder. A rigid arm cut at the shoulder will remain a rigid arm after rotation.

Create each character as a permanent asset package

Part	Assets to prepare
Head	Front and, if needed, 3/4 or profile views; stable hairstyle and face proportions.
Eyes / brows	Open, blink/closed, angry, surprised, relaxed and other key expressions.
Mouth	Neutral, closed, smile, frown, O, wide-open and phoneme/viseme shapes.
Torso	Base body plus separate clothing/outfit layers.
Arms	Left/right upper arms and forearms, with hidden overlap at shoulder and elbow.
Hands	Open palm, fist, pointing, gripping and phone-holding poses.
Legs / feet	Left/right thighs, shins and feet, with knee overlap and clear foot pivots.
Props	Phone, document, stamp, briefcase, football, coffee cup and other recurring objects.

Artwork preparation workflow

- Stage 1 — master design sheet: define face proportions, hairstyle, clothing, outline thickness, palette and front/3/4 reference views.
- Stage 2 — part extraction: generate or extract body parts using the master reference. Do not rely on one flattened AI sprite sheet as a production-ready rig.
- Stage 3 — joint repair: create hidden overlap geometry at shoulders, elbows, hips and knees so gaps do not appear during movement.
- Stage 4 — pivot alignment: set consistent pivots for shoulders, elbows, wrists, hips, knees, ankles, neck and head.
- Stage 5 — rig and validate: weight the mesh, test extreme poses and correct distortions before adding the character to the reusable library.

Character identity and outfit changes

Assign each character a permanent ID. Scene JSON should reference that ID rather than prompting an image generator to redraw the character for every scene. Swap clothing layers while retaining the underlying skeleton, body proportions, face and palette.

Example asset layout

```
characters/ manager_001/ reference.png palette.json rig.json head/ torso/ arms/ legs/ face/ hands/ outfits/
props/ animations/
```

5. Reusable action library driven by JSON

The AI should choose and parameterise known actions rather than inventing arbitrary movement descriptions for every scene. This makes animation repeatable, testable and reusable across characters.

Starter action library

Category	Initial actions
Locomotion	Walk, run, stop, turn, sit
Gestures	Wave, point, shrug, arms-crossed
Props	Use phone, hold paper, stamp document, pick up / put down object
Reactions	Shock, facepalm, laugh, cry, angry outburst
Talking	Idle talk, angry talk, excited talk
Transitions	Turn around, enter frame, exit frame, stumble

Example action JSON

```
{ "character": "manager_001", "action": "stamp_document", "start": 1.2, "duration": 1.8, "parameters": {
"hand": "right", "prop": "sacking_stamp", "target": "document", "impact_time": 0.85, "intensity": 1.2 },
"expression": "angry", "secondary_motion": { "torso": "lean_forward", "head": "follow_hand" } }
```

Use one shared skeleton naming convention

```
root ■■■ pelvis ■■■ torso ■ ■■■ neck ■ ■ ■■■ head ■ ■■■ upper_arm_L → forearm_L → hand_L ■ ■■■
upper_arm_R → forearm_R → hand_R ■■■ thigh_L → shin_L → foot_L ■■■ thigh_R → shin_R → foot_R
```

Make actions character-independent

Prefer a generic action such as walk(character_rig, speed, direction, emotion) rather than separate animations such as manager_walk, owner_walk and player_walk. The action player maps standard bone names to each character's rig.

Walk cycle quality checklist

- Contact: front foot touches down.
- Down: body absorbs impact.
- Passing: rear leg moves forward.
- Up: body rises.
- Repeat with the opposite leg; add hip movement, torso counter-rotation, arm swing, foot contact and head stabilisation.

Add procedural modifiers such as anticipation, overshoot, follow-through, shake and hold. These give the same basic action different comedic timing and personality.

Action channels prevent animation conflicts

Channel	Controls
Locomotion	Pelvis, legs and feet
Upper body	Torso and spine
Left / right arm	Shoulder, elbow, wrist and hand
Head	Neck and head
Face	Eyes, brows and mouth

This lets compatible actions run together—for example, walk + use_phone + blink + look_nervous—while preventing two actions from fighting over the same hand or arm.

6. Facial expressions and lip-sync

Use facial sprites, facial controls and mouth-shape switching rather than AI-generated talking-head video. Keep expression controls separate from speech mouth shapes so a character can talk while angry, crying or laughing.

Expression	Eyes	Eyebrows	Mouth
Neutral	Open	Normal	Closed
Happy	Relaxed	Raised	Smile
Angry	Narrow	Down	Frown
Shocked	Wide	Raised	Open
Crying	Closed / wet	Raised inward	Sad
Laughing	Closed	Raised	Wide smile

Recommended lip-sync utility: Rhubarb Lip Sync

- Open-source MIT-licensed command-line tool.
- Generates mouth-shape timings from audio.
- Can be integrated into the Python pipeline and mapped to character mouth sprites.
- Use a simplified set of approximately 5–7 mouth shapes for stylised cartoon speech.

Language limitation

Automatic recognition quality should be tested for Hindi dialogue. Do not assume phoneme alignment will be accurate. Fallback options include manually supplied timings, volume-driven mouth movement, or simplified mouth-open / mouth-closed animation.

A useful production shortcut: do not animate every character's mouth constantly. In short satirical reels, a static mouth with a blink, expression change and well-timed speech bubble can look better than inaccurate lip-sync.

7. Example scene sequence

Scene	Action choreography	Comedy beat
1. Manager arrives	Walk into training ground; look around.	Caption: “Another day at the office…”
2. Owner signs papers	Hold document; stamp it; smirk.	Stamp impact SFX.
3. Phone notification	Raise phone; read; eyes widen; shocked expression.	Notification SFX.
4. Replacement manager	Walk into away dugout; point / look nervous.	First game away at Anfield.

The AI writes the choreography; the renderer executes known actions. Scene descriptions should select assets, actions, expressions, props, timings and camera behaviour.

8. Scene JSON and validation

Extend the current JSON from describing only what the scene looks like to describing what each character does. A simplified schema:

```
{ "video": { "width": 1080, "height": 1920, "fps": 30, "duration": 30 }, "scenes": [{ "id": "scene_01",
"duration": 6, "background": "training_ground", "characters": [{ "id": "manager", "asset": "manager_001",
"position": [540, 1300], "outfit": "training_tracksuit", "actions": [ {"type": "walk", "start": 0, "duration":
3, "parameters": {"steps": 4, "speed": 1}}, {"type": "look_around", "start": 3, "duration": 1}, {"type":
"use_phone", "start": 4, "duration": 2} ], "expression": [ {"type": "neutral", "start": 0}, {"type": "shocked",
"start": 4.5} ] }], "captions": [ {"start": 0.5, "text": "Another day at the office..."} ], "sound_effects": [
{"type": "footsteps", "start": 0.2}, {"type": "notification", "start": 4.5} ] }] }
```

Validation before rendering

- Does the character ID exist?
- Does the outfit exist for that character?
- Does the action exist, and are its required props available?
- Are simultaneous actions compatible, or do they conflict over a channel?
- Are scene durations, dialogue timestamps and SFX timestamps valid?
- Does the full timeline match the requested reel duration?
- Are all referenced assets present and all output settings supported?

This validation layer prevents an AI script from producing a scene that the renderer cannot execute. Fail early with a useful error rather than producing a broken MP4.

Suggested project structure

```
football-cartoon-engine/ characters/ actions/ locomotion/ gestures/ props/ reactions/ scenes/ renderer/
project.godot main.tscn scripts/ action_player.gd character_controller.gd scene_director.gd
camera_controller.gd audio/ sfx/ ambience/ scripts/ validate_scene.py build_video.py output/
```

9. Quality, speed and cost expectations

The following are rough engineering planning estimates, not benchmark measurements or guaranteed service levels. Actual render times depend on frame rate, scene complexity, GPU availability, model configuration and export route.

Method	Indicative time for a 30-sec reel	Consistency	Main limitation
PIL + FFmpeg stills	Seconds to minutes	High	Slideshow effect
2D rig + Godot	Minutes or less to several minutes	Very high	Initial rig and action-library work
Rig + baked sprites	Minutes or less	Very high	Less flexible for new poses
AI motion transfer	Variable; minutes per shot possible	Medium	Artifacts and control limits
Wan 2.2	Potentially tens of minutes to hours	Low–medium	VRAM and shot consistency
LTX models	Highly configuration-dependent	Low–medium	Hardware and model requirements
Hybrid	Minutes to tens of minutes	High	More engineering components

Cost strategy

The marginal software and compute cost can potentially be ₹0 if you use open-source software, existing free compute allowances, locally stored assets and appropriately licensed free audio. That does not mean unlimited free compute: GitHub Actions and Kaggle have quotas, session limits and resource constraints.

- Use CPU for skeletal animation and scene compositing where practical.
- Reserve the Kaggle T4 allocation for AI asset generation and optional experiments.
- Cache character assets, backgrounds, action renders and audio so repeated reels do not regenerate them.
- Do not build a production plan that assumes free cloud GPUs are always available.

10. Licensing, likeness and platform risks

Software and model licensing

Tool / model	Licence / risk to verify
Godot	MIT; retain required licence notices.
Blender	GPL; commercial use of created artwork is permitted.
Rigby2D	Check the exact release, dependencies and export/runtime licences.
Rhubarb Lip Sync	MIT.
SAM 2	Apache-2.0; check model and dependency terms.
LTX-Video	Code and weights can have distinct terms; verify the exact checkpoint.
Wan 2.2	Verify the particular model/checkpoint terms and any use restrictions.
Wav2Lip	Research/non-commercial restrictions may apply; avoid monetised use without permission.
Viggle	Free-service commercial-use restrictions may apply; check current terms.

Always distinguish the licence for code from the licence for model weights, datasets, generated assets and hosted services. Free access does not automatically mean commercial permission.

Real public-figure caricatures

- Satirical caricature is different from impersonation or presenting fabricated footage as genuine, but it is not blanket legal clearance.
- Avoid depicting a person as committing a specific crime or dishonest act unless the context is clearly fictional and legally reviewed.
- Avoid implying endorsement by a person, club, league or sponsor.
- Be careful with personality/publicity rights, defamation, trademarks, club crests, sponsor logos, kit designs and copied cartoon styles.
- Use original character designs and an original visual style. Keep a clear distinction between commentary and claims about real events.

This is not legal advice. The legal position can depend on the exact depiction, context, commercial use and jurisdiction, including Indian law.

Instagram and YouTube

- Meta and YouTube have AI disclosure rules for qualifying realistic or materially altered content. Clearly unrealistic cartoon animation is generally treated differently from realistic synthetic footage; check current platform guidance.
- YouTube's inauthentic-content monetisation policy is relevant to mass-produced, repetitive template videos. Make each reel meaningfully original through distinct jokes, dialogue, story situations, character interactions and editing.
- Do not publish near-identical videos at scale with only names or captions swapped.

11. One-day prototype plan

Prototype goal

Build one football-manager character who walks into a room, receives a phone notification, looks shocked and performs a facepalm. This tests rigging, action playback, props, facial expressions, scene composition and headless rendering.

Time	Task	Deliverable
Hour 1	Install and inspect Rigby2D; import demo; inspect AutoRig and export options.	Verified export sample
Hours 2–3	Create one original character, prepare layers, rig elbows/knees/shoulders and add neutral, shocked and angry expressions.	First reusable character rig
Hours 4–5	Create walk, use_phone and facepalm actions; add blink and breathing.	Three playable actions
Hour 6	Create a Godot test scene with background, character placement and sequential actions.	Animated test scene
Hour 7	Read a simple JSON scene from Python; trigger Godot; render frames and encode with FFmpeg.	Automated MP4
Hour 8	Check seams, foot contact, phone attachment, identity consistency, transitions and unattended execution.	Go / no-go findings

Success criteria

- Elbows and knees bend without obvious gaps or rigid one-piece limbs.
- Feet look grounded during the walk cycle.
- The phone stays attached to the hand through the interaction.
- The character looks like the same character throughout.
- Expressions change at the intended timestamps.
- The scene renders without manual editor interaction.

Do not judge the first prototype primarily on artwork polish. Judge whether the character can perform multiple actions without losing identity or exposing seams.

12. Final recommendation and development order

Priority	Build	Why
1	Character asset standard	Establish identity and reusable artwork.
2	Reusable skeletal rig	Enable genuine joint movement.
3	Action library	Make choreography repeatable and composable.
4	Godot renderer	Enable headless production and composition.
5	Facial expressions and lip-sync	Add personality and dialogue.
6	Optional AI motion generation	Handle occasional special shots.

Recommended stack: Rigby2D + Godot + Python + Rhubarb Lip Sync + FFmpeg.

The biggest investment is not rendering technology. It is the reusable character and action library. Once that exists, each new reel should mostly require new dialogue, scene arrangements, backgrounds, props and

action combinations.

Three things to avoid

- Do not build the entire animation engine around image-to-video models; consistency and precise choreography are too important.
- Do not automatically slice flattened AI artwork and expect a production-quality rig; prepare layered artwork with joint overlaps.
- Do not create separate copies of every action for every character; standardise the skeleton and reuse the action library.

Key uncertainty to resolve first

Validate that Rigby2D's exported rigs can be played and controlled in your exact Godot headless environment. If the native runtime integration is difficult, test baked sprite-sheet exports in Godot before abandoning the tool. This is the highest-value technical risk to test on day one.

Implementation note: Tool features, free-tier limits, model licences and platform policies change. Verify the current official documentation and exact release terms before production or monetisation.